

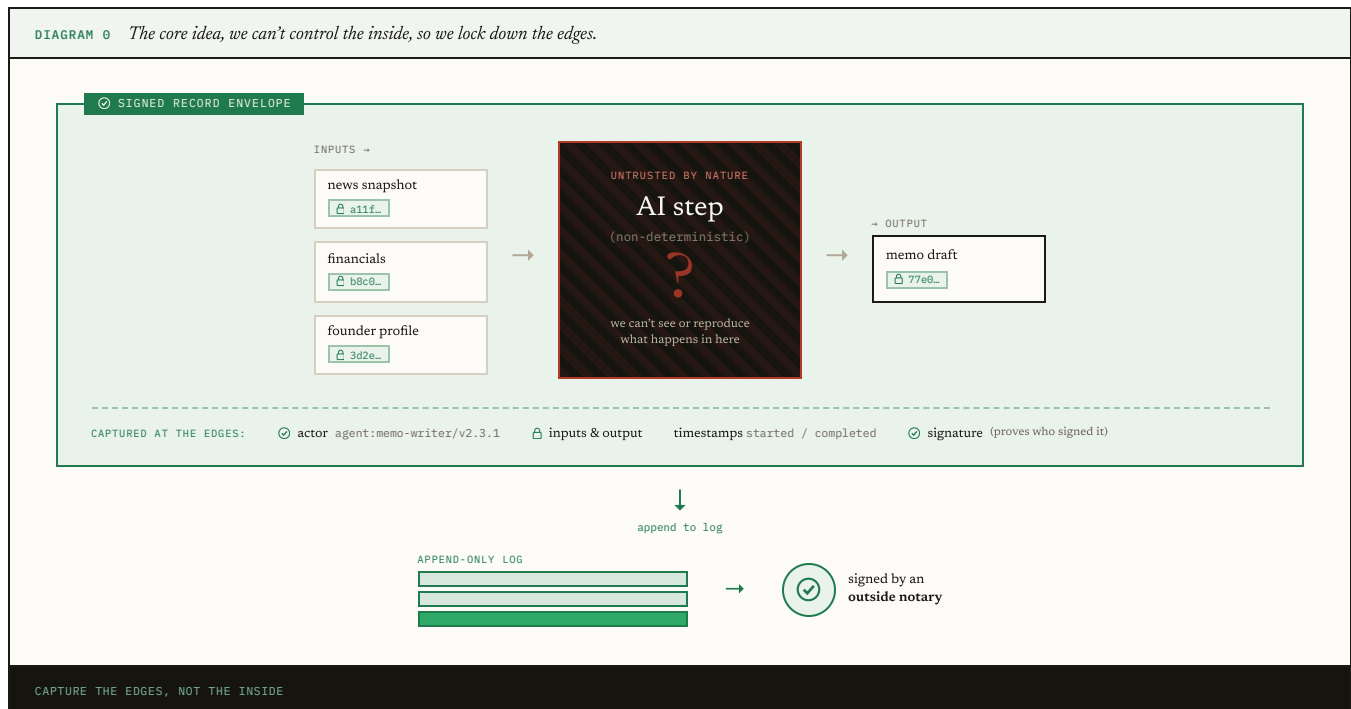
Dominic Lim *Soma Capital case study*

Approach

The design rests on one decision. You cannot make an LLM deterministic, so the goal is to make the *record* of what it did deterministic instead. The reason is temperature: unless a model is pinned to temperature zero it never returns the same output twice, so a step can never be retraced by rerunning it. The model stays unpredictable inside the step, but the moment anything crosses the step's edge, meaning the inputs it read, the output it produced, who ran it, and when, the system captures it exactly, signs it, and appends it to a log that cannot be rewritten. The inside stays sealed; everything observable from outside is exact, attributed, and permanent.

An ordinary database is append-only by policy, so an admin can edit a row and cover their tracks. This design is append-only by math. Records are chained by hash, so any change to any record cascades up to a single fingerprint at the top, which reveals the tampering wherever it happened, and that top fingerprint is signed by an outside party on a schedule, so checking it needs no trust in Soma at all.

At the brief's projection of 50,000 actions a day, roughly one write per second, throughput sits orders of magnitude below what ordinary databases and disks handle, so the focus belongs on correctness and auditability. The hard part is trust: the guarantee has to hold against a compromised server, a dishonest insider, and an auditor with no reason to believe anything Soma says. The whole architecture is built around surviving those three.



1 High-Level Architecture

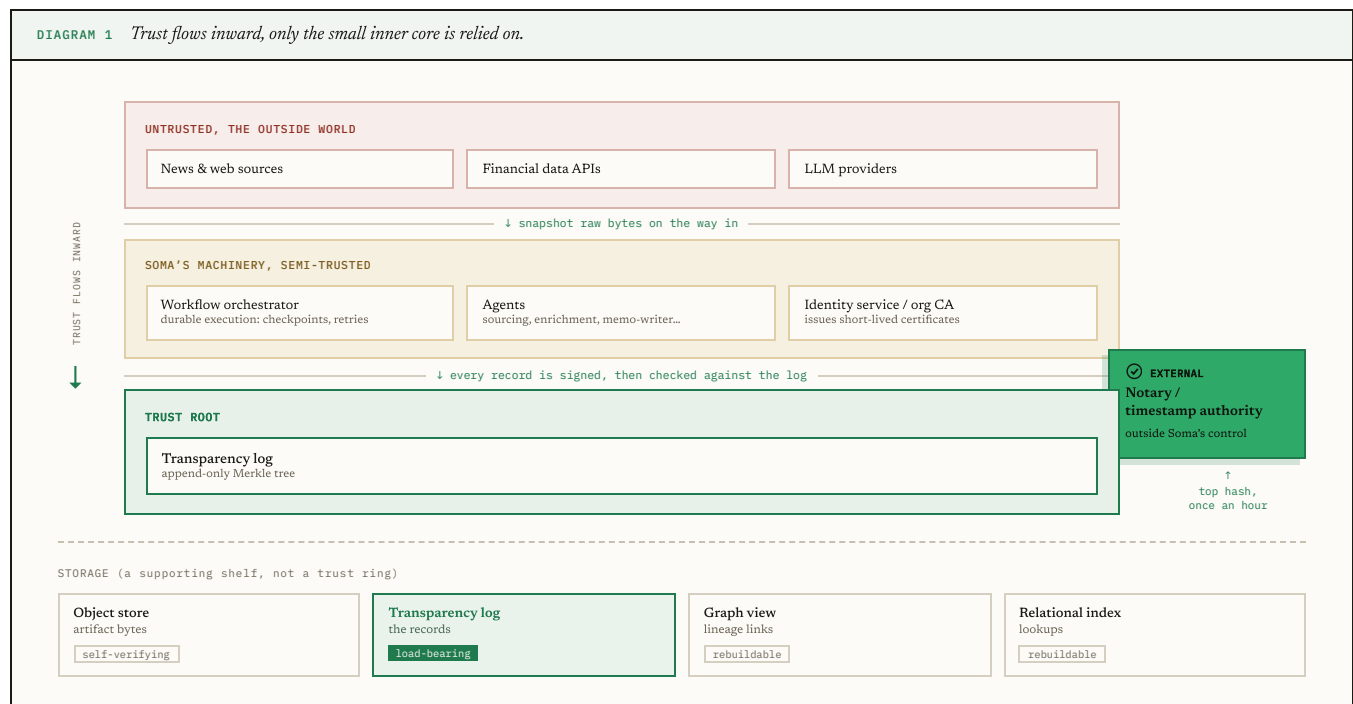
Requirements

The system has to prove whether information was modified, by whom, what was modified, and when. Each component below follows from one of those requirements rather than being chosen for its own sake. "Has this been modified" means an artifact must be identified by its own contents, which gives a **content-addressed store** where an artifact's address is a fingerprint of its content, so any change produces a different address. "Who made this, from what, and when" means every step needs a small signed **record** of its actor, inputs, and output. For records to be tamper-evident they cannot sit in a database an admin can edit, so they live in an append-only **transparency log**, and because even that log's operator could rewrite it, an **external anchor** pins its state somewhere Soma cannot reach.

The rest follow the same way: actors need identities and keys (the **identity service**), agents fail and retry across long runs without corrupting the record (a durable **workflow orchestrator**), and answering an auditor means walking records back into something they can check (the **audit layer**). The deeper reason for deriving them this way is that everything hangs off a single trusted source. Rather than build separate credibility for each of the five provenance answers, the design roots them all in the same anchored log, so the five come out as corollaries of one thing an outsider can check.

Trust boundaries

What you are allowed to trust is the whole game, since a guarantee is only as strong as its weakest dependency. The answer is three rings.



The outer ring is everything Soma does not control: news sites, financial APIs, LLM providers. It can be wrong, slow, or hostile, so nothing it says is taken as true; the system snapshots exactly what comes back and treats that as a record of what was received, not a fact about the world. The middle ring is Soma's own machinery, the orchestrator, agents, and identity service. It does the real work and holds signing keys, and the point is to avoid trusting it: assume any single machine could be compromised, and require that a lie told

here is caught by the inner ring. The inner ring is the trust root, the append-only log and the external notary, kept small because it is the one thing everyone relies on and a small root is one you can reason about end to end. Trust flows inward. Keeping the root this small is also what makes it sellable: for an LP the whole pitch collapses to one anchored number they can check themselves, and on an already hard sell you want one strong hook rather than many claims to defend.

Following one memo through the system

To see the pieces work together, follow one memo from raw sources to an approved decision, the exact path the prototype runs. A sourcing agent fetches a news article and a financials response and saves the raw bytes of each. Those two saves are the first records, and having no inputs makes them the frontier, the edge where outside data entered. An enrichment agent reads both and writes a founder profile; its first attempt times out, and that failure is written down as its own record, with the successful retry pointing back at it. A memo agent drafts from the profile, a partner edits the draft (creating a new document rather than overwriting the old one), and the approval pins the exact bytes approved. When the LP later asks the headline question, the system starts at that approval and walks the input links back to the two frontier snapshots.

Durable workflows resistant to failure

There is no need to rebuild the mature infrastructure for reliable execution. A durable engine such as Temporal already checkpoints progress so a crash resumes, and uses idempotency keys so a retried write happens once. The part that matters here is retries, which raise reliability even when a step is non-deterministic and give a natural checkpoint at which to capture provenance. The two kinds of step are handled in opposite ways. A deterministic step, a database write or an API call, can safely run again, since a second run has the same effect and the idempotency key absorbs the duplicate. A non-deterministic step, an LLM call, cannot be rerun to check it, because a second run gives a different answer, so its output is recorded once and reused rather than re-executed. Every attempt, including the one that failed, becomes its own permanent record, and the retry links back to the failure, so a history that drops that failure fails verification. The boundary is enforced by math, not policy.

Identity and the trust model

A signature only means something if you know whose key produced it and can be sure it was not quietly stolen months ago. The common approach, long-lived keys plus a revocation list, is where these systems fail: a stolen key stays valid until a human notices. So an organization CA issues short-lived certificates, good for hours, to each agent instance. Rotation takes care of itself, since an agent requests a fresh certificate near expiry and there is no long-lived secret to steal. The cost is an always-on issuer: if it is down, agents cannot sign. Keeping an issuer up is far easier than keeping a revocation list correct and fast; for an identity that had to sign offline, long-lived keys would win. There is a second reason I favor this beyond security: validation robust enough to make keys ephemeral is itself a strong point in front of LPs. Two more rules finish the model. An agent never acts on its own authority, so its certificate must carry a delegation chain ending at a real human or verification rejects it. And holding a valid key is not enough to sign as someone else, because the identity claimed in a record must match the certificate the CA issued. The model in one line: trust the CA for who is who, an outside notary for when history existed, and nothing else, including Soma's own servers.

Storage

There are four storage layers, and only one has to be trusted. Artifact bytes sit in ordinary object storage, self-verifying because the address is the hash. The records sit in the append-only log, which with the external anchor is the one load-bearing layer. A graph view speeds the audit walk and a relational index answers

everyday lookups, and both are derived: an auditor trusting none of Soma's databases could throw them away, rebuild from the log, and every proof would still hold. That leaves the brief's first requirement, five questions each artifact must answer, which the record fields below cover directly.

2 Deep Dive: the record, the DAG, and the log

This section goes deep on the provenance record, which is where "who made this, from what, and whether it has changed" is actually answered, and where a design that looks fine can be quietly wrong. That is why it leads the write-up: the rest of the system matters, but the notary and the log are where everything is verified, since without them you could rebuild the Merkle tree and claim it looked that way all along. Three things do the work: the record that captures a step, the hash-links that turn records into lineage, and the log that makes the history complete and impossible to rewrite.

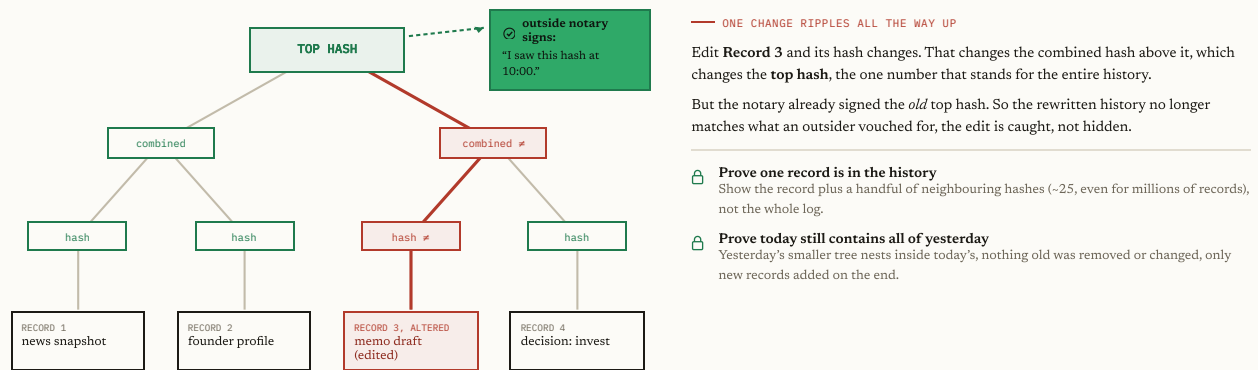
What a record captures, and why

The five questions decide the contents. "Who made this" calls for an actor identity and a signature over the whole record. "From what inputs" calls for a list of the exact upstream artifacts, each by content fingerprint. "Which version" puts the agent's version in its name, plus, for an AI step, the model and its settings. "When" calls for start and completion times. And "has it been modified" comes free from naming the output by fingerprint, because any change breaks every record that pointed at it. Two choices do a lot of work. The record names inputs and outputs by fingerprint instead of storing bytes, so it stays about a kilobyte and deleting an artifact for a privacy request never disturbs the records that point at it. And a failed attempt is a full record the retry links back to, so hiding a failure is structurally impossible. One field exists purely for the AI problem: the system cannot record why a model produced an output, but it records everything that went in, the model and version, the exact prompt, the settings, and the provider's request ID. The step is fully attributable even though it can never be reproduced, and that gap between attributable and reproducible is the honest center of putting AI in an audit trail.

Hash-links are the lineage, and the log makes it complete

Each record names its inputs by fingerprint, and each input was an earlier record's output that named its own inputs the same way, back to the frontier. The records form a graph whose edges are fingerprints sealed inside signed records. Faking an ancestor would need either two files with the same fingerprint, infeasible by design, or a forged signed record, which cannot be slipped into the log after the fact. Lineage and tamper-resistance become one mechanism. But a signature only stops a record from being changed, not deleted, and not shadowed by a second invented history. So the records go into a structure where no one, not even the operator, can delete one or swap in a fake without it showing: hash every record, hash those hashes in pairs, and again, up to a single top hash that fingerprints the whole history at once. This is a Merkle tree, the same construction that keeps the world's website certificates honest under Certificate Transparency.

DIAGRAM 4 Change one record and the single number at the top changes, so no one can rewrite the past quietly.



Two proofs come out of the tree almost for free, exactly what an auditor needs. You can prove one record is in the history with just that record and about 25 neighboring hashes, not the whole log. And you can prove today's history still contains all of yesterday's, unchanged, with new records only added on the end. One gap remains: all of this assumes someone kept yesterday's top hash. So once an hour the current top hash goes to an outside timestamping notary that signs "I saw this hash at this time." From then on it is pinned where Soma cannot reach, and rewriting even one old record fails to produce a valid proof back to it. Rewriting history stops being forbidden by policy and becomes caught by arithmetic.

Answering the LP with a proof that needs no trust in Soma

The audit layer takes the approval artifact, walks the input links back to the frontier, pulls in failed attempts, and packages a proof bundle: every record with its inclusion proof, the current and last-anchored top hashes with a consistency proof between them, every certificate, the notary's receipt, and optionally the bytes. What makes it a proof rather than a report is that a standalone verifier checks all of it while trusting exactly two public keys, the CA and the notary, sharing no database or code with the system that produced it. A malformed bundle returns a failure instead of crashing, so a crash cannot be mistaken for a pass. The same verifier defeats the four ways someone would try to cheat: editing an approved memo breaks its fingerprint; forging a record under a lookalike CA fails to chain to the real CA; hiding a failed attempt leaves the retry pointing at a missing record; and rebuilding the log with one record swapped cannot produce a consistency proof back to the notary's hash.

Two places this kind of system goes quietly wrong

Two subtle failures are worth calling out, because both are failures of attribution, the exact property the system guarantees, and both stay invisible until someone lies. The first is timestamp handling: in JavaScript every comparison against an invalid date returns false, so a broken timestamp reads as inside a certificate's validity window and an expired certificate slips through. The guard is to reject unparseable times and fail closed. The second is the difference between signing an identity and binding it: checking the signature and certificate is not enough, because the identity claimed in the record must also match the certificate's subject, or any valid key holder could attribute work to someone else. The guard is an exact match on the claimed identity and delegation chain. The pattern is the same, the dangerous gaps are in the checks a verifier forgets to make, which is why the tests that matter most are the ones that try to lie to it.

3 Tradeoffs and Future Direction

Every choice traces back to one goal: verifiability by a party who does not trust Soma. The rest I gave up on purpose. Throughput was never in question at one write per second. Storage stayed boring, since boring storage is easier to trust than something clever. And audit queries may take the minute the brief permits, which lets indexes be rebuilt from the log rather than maintained as fragile always-correct ones.

Three forks are worth naming. A blockchain would remove the trusted log operator entirely, but a permissionless chain needs validators, which exposes the network and has to pay them to run, out of proportion for an internal tool whose rewrite window is already held to an hour by the notary. It stays an escape hatch: anchoring the top hashes into a public chain is one integration away if that requirement arrives. A separate lineage database plus an audit log would be simpler, but it splits lineage and integrity into two systems that can drift, leaving the auditor trusting the join; making the signed record stream itself be the lineage removes that seam. Re-running steps to verify them stays available for deterministic steps but cannot work for LLMs, since a second run differs and providers do not sign their outputs, so the design records the full call context now and can upgrade to provider-signed inference when it exists.

The limits are worth stating plainly. The system proves lineage, not truth: a hallucination is recorded with perfect integrity, which is why decision-grade artifacts require a human approval, putting an accountable person in every real decision chain. That reflects a deliberate choice, talked through with Alan: lean on heuristics for the bulk of the work and bring LLMs in only at gating points, since a smaller non-deterministic surface means fewer decisions riding on something no one can re-derive. A live, compromised endpoint can sign real lies with a valid key; short certificate windows and the anchored log contain that without removing it, and I scoped it down on purpose because this is an internal tool, where the threat model leans toward honest mistakes and insider edits rather than a hostile public. Outside data is trusted on first fetch and fingerprinted immediately, so a dispute reduces to what their API returned at 14:02 UTC. Record timestamps are self-claimed, with per-record countersigned timestamps from the notary as the ready fix.

On what I would build next: at 10x to 100x nothing in the trust core changes shape. The step I care about most follows the heuristics-first stance, pushing more of each workflow into deterministic, checkable steps and narrowing the LLM toward pure gating, so the share that can only be attributed keeps shrinking. Beyond that the plumbing scales without touching the guarantee: the log shards per fund or workflow class under a shared super-root, anchoring batches, cold bytes tier to archival while fingerprints stay hot, and the derived views move to a dedicated graph store because they are safe to rebuild. What you could prove to the LP on day one is what you can still prove on the ten-thousandth.